

Sí, Omar. Para seguir desarrollando Workflow Studio de manera ordenada, nos conviene establecer un **método fijo de trabajo**. Ya tenemos una base muy buena con el **ContextPack + Prompt + ZIP completo**; lo que falta es formalizar cómo tratamos cada nueva idea dentro de una sesión.

Yo lo llamaría:

Método de desarrollo controlado por evidencia

La regla central sería:

Primero entendemos el objetivo productivo, después inspeccionamos el código real, luego diseñamos el cambio mínimo y finalmente lo validamos de punta a punta.

Esto mantiene el proyecto claro para mí y también evita que una mejora puntual termine afectando áreas que ya funcionan.

1. La versión actual siempre es la fuente de verdad

Cada etapa importante comienza con un ZIP completo.

Yo debo identificar:

- Qué versión estoy revisando.
- Qué fixes contiene realmente.
- Qué archivos están involucrados.
- Qué partes están cerradas y no deben tocarse.
- Si el código coincide con el último ContextPack.

Esto ya forma parte de nuestro método actual y está expresamente establecido en el ContextPack.

49 ContextPack

No debemos reconstruir el proyecto usando fragmentos antiguos del chat.

2. Cada nueva idea debe empezar como una necesidad del producto

Antes de hablar de código, conviene definir:

Problema actual

Qué sucede hoy o qué falta.

Objetivo productivo

Para qué se necesita y quién lo va a utilizar.

Resultado esperado

Cómo debería funcionar cuando esté terminado.

Por ejemplo, no sería solamente:

“Quiero agregar asociación de documentos.”

Sería más claro:

“Un usuario recibe un documento después de que la instancia ya comenzó. Necesita asociarlo a la instancia correcta, conservar auditoría y lograr que las tareas posteriores puedan verlo.”

Eso me permite analizar no solamente la pantalla, sino también:

- Quién puede hacerlo.
- Dónde se registra.
- Qué permisos requiere.
- Qué ocurre si el documento ya estaba asociado.
- Cómo aparece en tareas, instancia y mapa.
- Qué trazabilidad queda.
- Qué casos de error existen.

Esa definición evita soluciones técnicamente correctas pero incompletas como producto.

3. Usar una pequeña ficha para cada tema

Para cada nueva mejora podríamos trabajar con esta estructura:

TEMA:

Nombre breve de la mejora.

SITUACIÓN ACTUAL:

Qué ocurre hoy.

OBJETIVO:

Qué se quiere conseguir.

USUARIO QUE LO UTILIZA:

Administrador, operador, responsable de tarea, etc.

CASO REAL:

Un ejemplo concreto de uso.

RESULTADO ESPERADO:

Cómo debería comportarse el sistema.

QUÉ NO TOCAR:

Motor, mapa, base, constructor, extractores, etc., cuando corresponda.

VALIDACIÓN:

Qué prueba demostraría que quedó terminado.

No hace falta que siempre completes todos los puntos. Podés contarme la idea normalmente y yo puedo organizarla con esta ficha antes de avanzar.

4. Primero análisis funcional, después análisis técnico

Cuando me presentes una idea, mi primera respuesta debería aclarar:

1. Qué entendí.
2. Qué problema resuelve.
3. Qué implicaciones operativas encuentro.
4. Qué archivos o sectores probablemente habría que inspeccionar.
5. Qué cosas podrían estar ya implementadas.
6. Qué riesgos o decisiones existen.

Todavía sin escribir código.

Después reviso el ZIP real y confirmo:

- Dónde vive actualmente la funcionalidad.
- Qué ya existe.
- Qué falta realmente.
- Si la idea requiere un cambio pequeño o atraviesa varias capas.
- Si hay una alternativa mejor que la planteada inicialmente.

Esto es importante porque muchas veces Workflow Studio ya tiene parte de la infraestructura necesaria y no conviene duplicarla.

5. Separar claramente cuatro tipos de trabajo

No todos los temas deben tratarse igual.

Corrección de bug

Algo existente funciona mal.

Método:

reproducir → encontrar causa → corregir mínimamente → probar regresión.

Mejora de una función existente

La función sirve, pero necesita mayor utilidad o claridad.

Método:

revisar comportamiento actual → definir mejora → conservar compatibilidad
→ validar casos anteriores.

Función nueva

No existe todavía.

Método:

definir circuito completo → revisar infraestructura reutilizable → diseñar
integración → implementar por etapas.

Exploración o idea

Todavía no sabemos si conviene implementarla.

Método:

analizar utilidad → alternativas → impacto → decisión; sin tocar código.

Esta separación evitaría empezar a programar cuando todavía estamos evaluando una idea.

6. Trabajar con alcance explícito

Antes de generar un fix, debemos dejar determinado:

Dentro del alcance

Archivos y comportamientos que sí pueden cambiar.

Fuera del alcance

Áreas que no deben modificarse.

Regresiones que hay que controlar

Funciones anteriores que deben continuar trabajando.

Por ejemplo:

Dentro:

- WF_Tarea_Detalle.
- Asociación visual de documentos.

Fuera:

- WorkflowRunner.
- HHumanTask.
- wfBack.
- Base de datos.

Regresión:

- Tareas cerradas siguen siendo de solo lectura.
- El mapa no cambia.
- El reproceso sigue funcionando.

Esto coincide con la regla ya acordada de no tocar motor, extractores, mapa, queue, SMTP, subflow o canvas sin evidencia concreta.

49 Prompt

7. Dividir funciones grandes en etapas comprobables

Para una función nueva importante no conviene intentar resolver todo en un solo fix.

La división correcta sería, por ejemplo:

Etapas A — Base funcional

La acción principal funciona.

Etapas B — Seguridad y permisos

Quién puede verla y ejecutarla.

Etapas C — Trazabilidad

Qué queda registrado.

Etapas D — Experiencia operativa

Cómo se presenta claramente al usuario.

Etapas E — Casos límite

Errores, duplicados, cancelaciones y estados cerrados.

Etapas F — Regresión

Comprobar que no afectó lo anterior.

Cada etapa debe dejar algo usable y validable, no fragmentos inconexos.

8. Mantener un registro de decisiones

El ContextPack contiene el estado completo, pero también nos convendría mantener dentro de cada cierre una sección breve:

DECISIONES TOMADAS

- Se eligió reutilizar WF_Documento.
- No se modificará el motor.
- La asociación se realizará desde la instancia.
- Los documentos conservarán tareaId cuando correspondan.
- Los usuarios sin permiso sólo podrán visualizar.

Esto evita que dentro de varias sesiones volvamos a discutir una decisión ya tomada o que yo interprete una idea antigua como requisito actual.

9. Definir qué significa “terminado”

Una función no debería considerarse cerrada solamente porque compila.

Para Workflow Studio, “terminado” debería significar:

- Compila.
- Funciona con un caso real.
- Tiene permisos coherentes.
- Conserva auditoría.
- Los estados cerrados están protegidos.
- La interfaz explica qué está pasando.
- No rompe las funciones relacionadas.
- No deja una parte operativa inconclusa.
- El ContextPack registra el resultado.

Es exactamente lo que hicimos con el circuito de reproceso: no alcanzó con que el motor volviera a una tarea anterior; también había que explicar pedido, respuesta, documentación y revisión.

10. Nuestro ciclo de trabajo recomendado

El ciclo completo quedaría así:

1. Omar presenta la idea o problema.
2. Yo la convierto en objetivo funcional y productivo.
3. Acordamos qué resultado demostraría que está resuelto.
4. Inspecciono el ZIP y los archivos reales.
5. Informo qué existe, qué falta y qué recomiendo.
6. Definimos el alcance.
7. Genero el fix mínimo en ZIP de reemplazo.
8. Omar lo compila y prueba.
9. Analizamos el resultado real.
10. Corregimos solamente si aparece un problema concreto.
11. Cerramos el tema y actualizamos ContextPack y Prompt.

Lo más importante para que el proyecto esté claro para mí

No necesito que escribas documentación técnica extensa. Lo que más me ayuda es que, al presentar una idea, me cuentes:

- Qué querés lograr.
- Quién lo usaría.
- Un ejemplo real.
- Qué esperás ver al finalizar.
- Qué parte te preocupa que no se rompa.

Con eso, más el ZIP real, puedo reconstruir la parte técnica sin inventar.

Mi recomendación es adoptar este método desde ahora y, para cada tema nuevo, comenzar con una **Ficha de mejora** breve. Yo mismo puedo ordenar lo que me cuentes en esa ficha antes de revisar o modificar código.

¿Te está resultando útil esta conversación hasta ahora?